

HANDS-ON BOOTCAMP · 40 MIN

Building a Claude Code Workspace

Structure a real project — codebase + Agents + Skills + MCP — and watch Claude become a teammate that already knows how it works.

Friday Bootcamp

Live demo + hands-on lab · Demo repo: [workspace_demo](#)





AGENDA

What we'll cover in 40 minutes



The mental model

code + the .claude layer



The folder map

every file, what it's for



CLAUDE.md & settings

memory + governance



Agents

specialist subagents



Skills

reusable workflows



MCP

connect external tools

You'll leave able to: lay out a workspace · add a Skill · add an MCP server · know what to commit vs. keep secret.



THE BIG IDEA

A workspace = code + an agent layer

Your repo doesn't change. A workspace adds a thin, version-controlled config layer on top — so a teammate who clones the repo gets a Claude that already knows the project.



Memory

CLAUDE.md — standards Claude reads every session



Capabilities

Agents & Skills — reusable units of behavior



Connections

.mcp.json — links to your tools & data



Governance

settings.json — what's allowed, enforced

the workspace layer ▲

CLAUDE.md + .claude/ + .mcp.json

settings.json · permissions · hooks

backend/ · frontend/ (your code)



THE DEMO

The app we'll build the workspace around

A small, real AI chat assistant. The app is deliberately simple — the point is the workspace wrapped around it. It even runs without an API key (demo mode), so the live demo can't fail on stage.



Backend Python · FastAPI · Anthropic SDK



Frontend plain HTML / CSS / JavaScript



The layer .claude/ + CLAUDE.md + .mcp.json

How a message flows

Browser frontend/app.js



POST /api/chat backend/main.py



Claude API backend/llm.py



reply → UI rendered in chat



THE MAP

The workspace, file by file

```
workspace_demo/
├── .claude/           # the config layer (committed)
│   ├── settings.json # permissions, env
│   ├── agents/       # subagents (code-reviewer...)
│   ├── skills/       # workflows (add-endpoint...)
│   └── commands/     # slash commands (/ship-check)
├── CLAUDE.md         # project memory, every session
├── .mcp.json         # external tool connections
├── .gitignore        # .env & *.local stay out of git
├── backend/         # FastAPI app (Python)
└── frontend/        # HTML / CSS / JS
```

✅ Committed & shared

.claude/, CLAUDE.md, .mcp.json — the team gets the same setup.

❌ Kept out of git

.env (your key) and *.local.json (personal overrides).

Mirrors at ~/.claude/

Same folders exist at user scope, for all your projects.



MEMORY

CLAUDE.md — context, every session

```
# Workspace Demo — AI Chat

## Commands
- run: `uvicorn backend.main:app`

## Conventions
- 4-space indent, type hints
- secrets in .env, never hardcode

## Capabilities
- agents: code-reviewer, api-tester
- skills: add-endpoint, run-app
```



Loaded up front

In context from session start — so keep it lean (< 200 lines).



Not just docs

Build commands, conventions, architecture — what Claude must know.



Generate it fast

Run /init and Claude scans the repo and drafts it for you.



Long stuff → Skills

Procedures belong in a Skill, not bloating CLAUDE.md.



GOVERNANCE

What's allowed · what's committed

settings.json — guardrails (committed)

```
"permissions": {  
  "allow": [ "Bash(uvicorn:*)" "  
            "Bash(pip install:*)" ],  
  "deny": [ "Read(./env)" ]  
},  
"env": { "PYTHONUNBUFFERED": "1" }
```

Lets Claude run safe commands without asking; blocks reading your key.

.gitignore — share vs. secret



Commit (team shares)

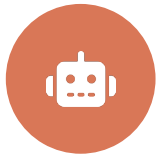
```
.claude/ · CLAUDE.md · .mcp.json  
· .env.example
```



Ignore (private / secret)

```
.env · .claude/settings.local.json ·  
__pycache__/
```

Precedence: managed › local › project (.claude/) › user (~/.claude/). Rule: commit .claude/, keep .env & *.local out.



AGENTS

Specialist subagents

```
--- .claude/agents/code-reviewer.md
name: code-reviewer
description: Reviews diffs for bugs
  & security. Use after edits.
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

```
You are a meticulous code
reviewer. Flag correctness and
security first; cite file:line.
```

This project ships two: code-reviewer + api-tester — one .md file each.

Own fresh context

Heavy work stays out of your main thread.

description = when

Tells the main Claude when to delegate here.

tools = sandbox

Scoped to read-only-ish; omit → inherits all.

model per job

Cheaper model for a bounded task.



SKILLS ★

Reusable workflows + how they scale

```
--- skills/add-endpoint/SKILL.md
name: Add API endpoint
description: Scaffold a route.
  Use when adding an endpoint.
---
1. Add Pydantic model + route
2. Wire frontend fetch()
See reference/conventions.md
```

A Skill = a folder with a SKILL.md (+ optional scripts/ & reference/).

Progressive disclosure — 3 levels

1

Metadata — always on

just name + description

2

Body — on trigger

full SKILL.md when it matches

3

Files — on demand

reference/ read only when opened

Why it matters: ship dozens of skills + huge reference docs without bloating context — Claude pays only for what it opens.



MCP

Connecting Claude to external tools

```
// .mcp.json (repo root, committed)
"mcpServers": {
  "filesystem": {
    "command": "npx"
  },
  "fetch": {
    "command": "uvx",
    "args": ["mcp-server-fetch"]
  }
}
```

Add via CLI

```
claude mcp add fetch -s project \
  -- uvx mcp-server-fetch
```

...or edit .mcp.json

add a server entry by hand

Then /mcp

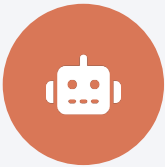
check status & authenticate (OAuth)

⚠ **Never commit tokens to .mcp.json.** MCP secrets go in env vars or your personal config.



DECISION

Agents vs. Skills vs. MCP



Agent

when you need isolation

- > Heavy / context-hungry work
- > A constrained role + scoped tools
- > A different model for the job



Skill

when you need a procedure

- > A repeatable, step-by-step workflow
- > Bundled scripts & reference docs
- > Auto-triggers by description



MCP

when you need the outside world

- > Reach a database, web, GitHub
- > Standard protocol for any tool
- > Shared via .mcp.json

They compose: a skill can run inside an agent; an agent can use MCP tools.



LIVE DEMO

Run the app

```
$ pip install -r requirements.txt
$ uvicorn backend.main:app --reload

→ open http://127.0.0.1:8000
```

- UI loads — type "hello", get a reply
- Status pill shows demo mode (or live · model)
- No key needed — safe on stage
- Point out: CLAUDE.md & settings already did the setup

Then (optional): ask Claude to add an endpoint → it uses the add-endpoint skill; or run the code-reviewer agent on the diff.

If it breaks: --port 8001, or show the curl output in the cheat sheet. Never debug live > 30s.



YOUR TURN · ~5 MIN

Hands-on: create your first Skill

1

Make the folder

```
mkdir -p .claude/skills/greet
```

2

Add SKILL.md

name + a one-line description (the trigger)

3

Use it

run `/greet` — or ask "welcome teammate Sara"

```
--- .claude/skills/greet/SKILL.md
name: Greet the user
description: Write a short,
            friendly welcome for a new
            teammate. Use to greet or
            onboard someone.
---

# Greet a new teammate
1. Welcome them by name.
2. Point to README + the guide.
3. Keep it 3-4 sentences.
```

No install, no restart — Claude auto-discovers the folder. Full steps in LAB.md.



RECAP

Five things to take home

1

A workspace = code + a committed `.claude/` layer

Your code is untouched; the layer teaches Claude the project.

2

CLAUDE.md is memory

Always loaded, high-level, lean (< 200 lines).

3

Agents isolate · Skills package · MCP connects

Pick by the job; they compose together.

4

Progressive disclosure makes it scale

Metadata always · body on trigger · files on demand.

5

Commit `.claude/` — keep `.env` & `*.local` out

Share the setup; never share secrets.

THANK YOU

Questions?

Now go build a workspace. Everything's in the repo:

- `README.md` — quick start
- `WORKSPACE_GUIDE.md` — full walkthrough
- `bootcamp/LAB.md` — hands-on labs
- `bootcamp/CHEATSHEET.md` — one-page reference